

DevOps

Detailed 5 Unit Semester Exam Notes

Theory, tables, diagrams and command/code examples

Watermark: Rahul yadav

Unit	Main Topics
Unit 1	DevOps basics, CI/CD, IaC, business drivers, lifecycle and benefits
Unit 2	DevOps framework, SCM, reviews, build/release, maturity and Agile/Cloud
Unit 3	Jenkins, GitHub, Maven, CI/CD best practices, Gerrit and testing
Unit 4	Build pipeline, Chef, Ansible, monitoring, Splunk/Nagios and cloud security
Unit 5	Docker, Kubernetes, Dockerfile, containers, dashboard, Helm and ecosystem

UNIT 1: DevOps Infrastructure

1. Meaning of DevOps

DevOps is a software engineering culture and practice that combines development and operations teams to deliver software faster, safer and more reliably. It focuses on collaboration, automation, continuous feedback and shared responsibility.

In traditional software delivery, developers write code and operations teams deploy it separately. This separation often causes delays, communication gaps and deployment failures. DevOps removes this gap by using automation, CI/CD pipelines, monitoring and infrastructure as code.

- DevOps is not only a tool; it is a culture, process and automation approach.
- It improves collaboration between developers, testers, security and operations teams.
- It supports frequent releases and faster feedback.
- It reduces manual errors by automating build, test, deployment and infrastructure.

2. CI, CD and Continuous Deployment

Concept	Detailed Meaning	Example
Continuous Integration	Developers frequently merge code into shared repository. Automated build and tests are triggered on every change.	Git push triggers Jenkins build
Continuous Delivery	Code is always kept ready for production release after automated testing	Release is approved manually
Continuous Deployment	Every successful pipeline change is automatically deployed to production	Auto deploy after tests pass



3. Infrastructure as Code

Infrastructure as Code means managing servers, networks, databases and cloud resources using code instead of manual setup. IaC files define what infrastructure should exist, and tools create or update it automatically.

IaC makes infrastructure repeatable, version controlled and easier to audit. It is important for cloud computing because cloud resources can be created and destroyed quickly. Common IaC tools include Terraform, CloudFormation, Ansible, Chef and Puppet.

- Infrastructure is written as configuration files.
- Changes can be tracked using Git.
- Same environment can be recreated for dev, test and production.
- Manual configuration errors are reduced.
- IaC supports disaster recovery and scalability.

4. Business Drivers for DevOps Adoption

Modern businesses need fast software delivery because customer expectations are high and market competition is strong. Data explosion, cloud computing, big data, data science and machine learning have increased the need for scalable and automated infrastructure.

DevOps helps organizations release features quickly, fix bugs faster, improve reliability and reduce operational cost. Cloud computing provides flexible infrastructure, while DevOps provides the process and automation to use it effectively.

- Need for faster release cycles.
- Need for better software quality and reliability.
- Growth of cloud, big data and AI/ML workloads.
- Requirement for automation and repeatable deployments.
- Pressure to reduce downtime and operational cost.

5. DevOps Lifecycle

The DevOps lifecycle is a continuous loop where software is planned, coded, built, tested, released, deployed, operated and monitored. Feedback from monitoring is used for the next planning cycle.



Point	Traditional Operations	DevOps
Team Structure	Separate development and operations	Collaborative cross-functional teams
Deployment	Manual and less frequent	Automated and frequent
Feedback	Slow feedback after release	Continuous feedback through monitoring
Infrastructure	Manual configuration	Infrastructure as Code
Risk	High release risk	Small frequent releases reduce risk

UNIT 2: DevOps Framework

1. DevOps Process

A DevOps process defines how teams move code from idea to production using automated and controlled steps. It begins with planning and source code management, continues through build and test automation, and ends with release, deployment, monitoring and feedback.

The process is iterative. After deployment, monitoring data and user feedback are used to improve the next release. This creates a culture of continuous improvement.

- Plan requirements and define tasks.
- Manage source code using Git or other SCM tools.
- Review code for quality and security.
- Build and package software artifacts.
- Run automated tests.
- Release and deploy using automated pipelines.
- Monitor application and infrastructure.

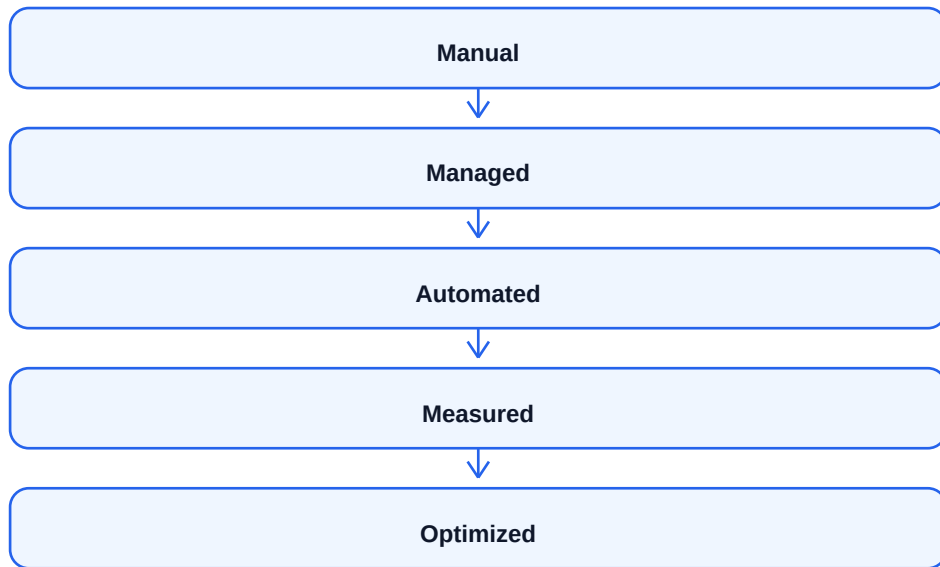
Area	Meaning	Tool Examples
Source Code Management	Stores and tracks code changes	Git, GitHub, GitLab, Bitbucket
Code Review	Checks code quality before merge	Pull Requests, Gerrit
Configuration Management	Manages server configuration	Ansible, Chef, Puppet
Build Management	Compiles/packages application	Maven, Gradle, npm
Artifact Repository	Stores build packages	Nexus, Artifactory, Docker Hub
Release Management	Controls release planning and approvals	Jenkins, GitLab CI, Azure DevOps
Test Automation	Runs tests automatically	JUnit, Selenium, PyTest

2. DevOps Maturity Lifecycle

DevOps maturity describes how advanced an organization is in using DevOps culture, automation and measurement. At low maturity, work is manual and teams are separated. At high maturity, teams use automated pipelines, IaC, monitoring and continuous improvement.

A maturity map helps organizations identify their current stage and plan improvements. It includes areas such as culture, source control, testing, deployment, infrastructure, security and monitoring.

- Level 1: Manual processes and separate teams.
- Level 2: Source control and basic build automation.
- Level 3: Automated testing and CI pipeline.
- Level 4: Automated deployment, IaC and monitoring.
- Level 5: Continuous improvement, security automation and self-service platforms.



3. DevOps Maturity Checklist

- Is all source code stored in Git or another SCM?
- Are code reviews mandatory before merge?
- Does every commit trigger an automated build?
- Are unit, integration and security tests automated?
- Are artifacts versioned and stored in a repository?
- Is infrastructure managed using code?
- Are applications monitored after deployment?
- Is rollback strategy defined?

4. Agile Framework and Cloud as Foundation

Agile and DevOps are closely related. Agile improves development by using short iterations, customer feedback and adaptive planning. DevOps extends Agile by improving build, test, release, deployment and operations.

Cloud computing acts as a foundation for DevOps because resources can be provisioned quickly. Cloud platforms provide virtual machines, containers, managed databases, storage, networking, monitoring and scaling features. DevOps uses these cloud services through automation and IaC.

- Agile focuses on iterative development and collaboration.
- DevOps focuses on delivery pipeline and operations automation.
- Cloud provides scalable and on-demand infrastructure.
- Together Agile, Cloud and DevOps enable fast software delivery.

UNIT 3: DevOps as a Service - CI, Delivery and Deployment

1. Best Practices for CI/CD

CI/CD best practices help teams deliver software safely and frequently. A good pipeline should be fast, reliable, repeatable and visible to the team. Every code change should be built and tested automatically.

Small commits are preferred because they are easier to review and debug. Tests should run at multiple levels such as unit, integration and acceptance. Artifacts should be versioned so the same package can be promoted across environments.

- Commit code frequently in small changes.
- Run automated build and test on every commit.
- Keep pipeline fast and reliable.
- Store artifacts in a repository.
- Use environment-specific configuration.
- Automate rollback and monitoring.

2. Jenkins Setup and Tool Integration

Jenkins is an open-source automation server used to create CI/CD pipelines. It integrates with GitHub for source code, Maven for Java builds, testing tools for automation and deployment scripts for release.



Integration	Purpose
Git/GitHub with Jenkins	Pull source code and trigger builds using webhook
Maven with Jenkins	Compile Java project, run tests and create package
Jenkins Jobs	Define build steps and post-build actions
Gerrit	Code review before merge
Testing with Jenkins	Run unit, integration and UI tests automatically

3. Sample Jenkins Pipeline

```
pipeline {
  agent any
  stages {
    stage('Checkout') {
      steps { git 'https://github.com/user/project.git' }
    }
    stage('Build') {
      steps { sh 'mvn clean package' }
    }
    stage('Test') {
      steps { sh 'mvn test' }
    }
    stage('Archive') {
      steps { archiveArtifacts artifacts: 'target/*.jar' }
    }
  }
}
```

4. Gerrit and Repository Management

Gerrit is a code review tool used with Git. It helps teams review code before it is merged into the main branch. Developers push changes to Gerrit, reviewers inspect the code, comments are added and only approved changes are merged.

Repository management involves organizing branches, protecting main branch, defining merge rules and maintaining version history. A good repository strategy improves collaboration and reduces production defects.

- Gerrit supports peer code review.
- It integrates with Git repositories.
- Reviewers can approve or reject changes.
- Jenkins can run tests on Gerrit changes before merge.
- Repository management improves traceability and release control.

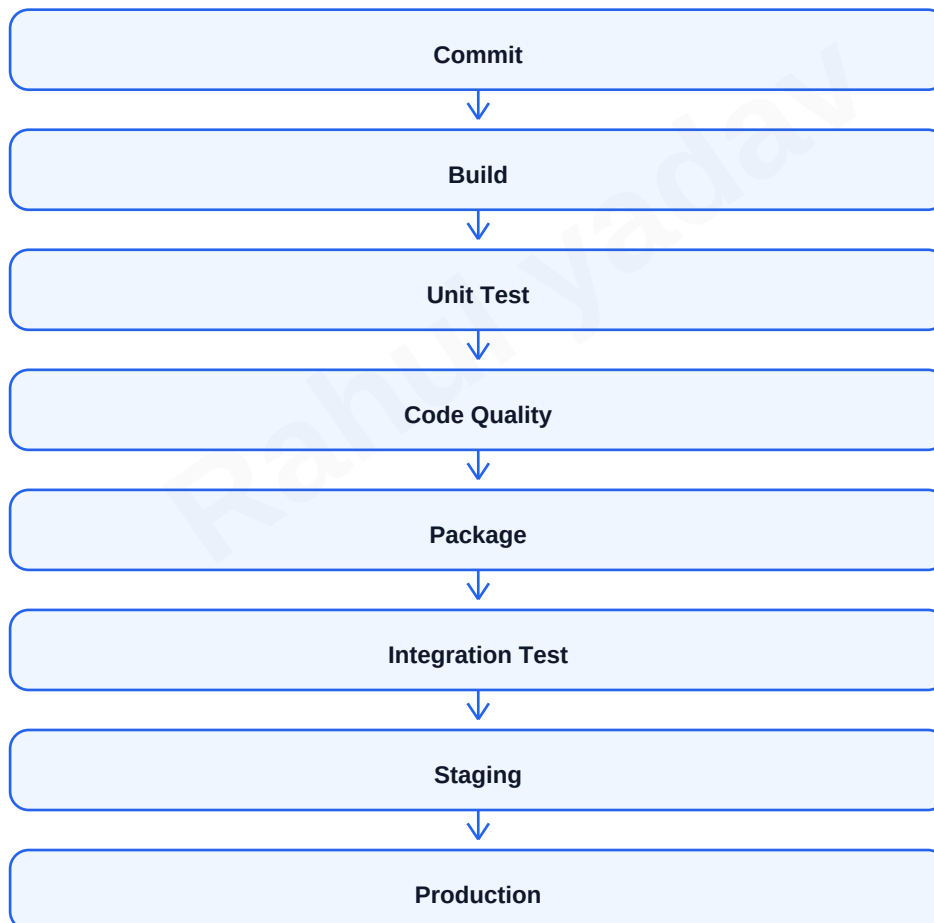
UNIT 4: Continuous Delivery, Configuration Management and Monitoring

1. Continuous Delivery and Build Pipeline

Continuous Delivery ensures that software is always in a releasable state. A build pipeline automates the steps from source code to deployment. It usually includes compilation, unit testing, code quality checks, packaging, integration testing and deployment to staging.

Continuous Deployment goes one step further by automatically deploying every successful change to production. This requires strong test automation, monitoring and rollback mechanisms.

- Build pipeline gives visibility into release status.
- Each stage acts as a quality gate.
- Failed pipeline stops bad code from moving forward.
- Continuous deployment needs high confidence in automation.



2. Chef Landscape and Chef Automate

Chef is a configuration management tool used to automate infrastructure configuration. It uses cookbooks and recipes to define desired server state. Chef Server stores configuration, Chef Workstation is used by developers/admins, and Chef Client runs on nodes.

Chef Automate provides workflow, compliance, visibility and automation features. It helps manage infrastructure at scale and ensures consistency across environments.

- Chef Workstation: where cookbooks are developed.

- Chef Server: central store for cookbooks and node data.
- Chef Client: runs on managed nodes and applies configuration.
- Cookbook: package of configuration code.
- Recipe: specific configuration instructions.

```
# Simple Chef recipe example
package 'nginx' do
  action :install
end

service 'nginx' do
  action [:enable, :start]
end
```

3. Ansible Features and Components

Ansible is an agentless configuration management and automation tool. It uses SSH to connect to managed nodes and YAML playbooks to define automation steps. It is popular because it is simple, readable and does not require agents on target machines.

Ansible inventory defines target hosts. Modules perform tasks such as installing packages, copying files or managing services. Playbooks organize tasks. Plugins extend functionality. Ansible Vault stores secrets securely, and Ansible Galaxy provides reusable roles.

- Inventory: list of managed hosts.
- Playbook: YAML file containing automation tasks.
- Module: reusable unit that performs one operation.
- Plugin: extends Ansible behavior.
- Ansible Tower/AWX: web UI and enterprise control.
- Vault: encrypts passwords and secrets.
- Galaxy: community roles and collections.
- Ansible CMDB: generates configuration reports from facts.

```
---
- name: Install and start nginx
  hosts: webserver
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
        enabled: yes
```

4. Monitoring and Cloud Security Issues

Monitoring is essential in DevOps because it provides visibility into application and infrastructure health. Monitoring tools collect metrics, logs and alerts. Nagios is used for infrastructure monitoring. Splunk is used for log analysis and searching machine data.

Cloud security issues include misconfigured storage, weak access control, exposed secrets, insecure APIs, lack of encryption and poor monitoring. DevOps teams use DevSecOps practices to integrate security into the pipeline.

- Nagios monitors servers, services and network devices.

- Splunk analyzes logs and machine data.
- Alerts help teams react quickly to failures.
- Cloud security requires IAM, encryption, network security and secret management.
- DevSecOps adds security testing into CI/CD.

Rahul yadav

UNIT 5: Containerized Applications with Docker and Kubernetes

1. Docker and Containers

Docker is a platform used to build, package and run applications in containers. A container includes application code, dependencies, libraries and runtime environment. Containers are lightweight compared to virtual machines because they share the host operating system kernel.

Docker helps solve the problem of 'it works on my machine'. If an application is packaged as a container image, it can run consistently on developer laptop, test server and production environment.

- Dockerfile defines how to build an image.
- Image is a read-only package of application.
- Container is a running instance of an image.
- Docker Hub is a public image registry.
- Containers are portable, lightweight and fast to start.

2. Dockerfile Example

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

3. Common Docker Commands

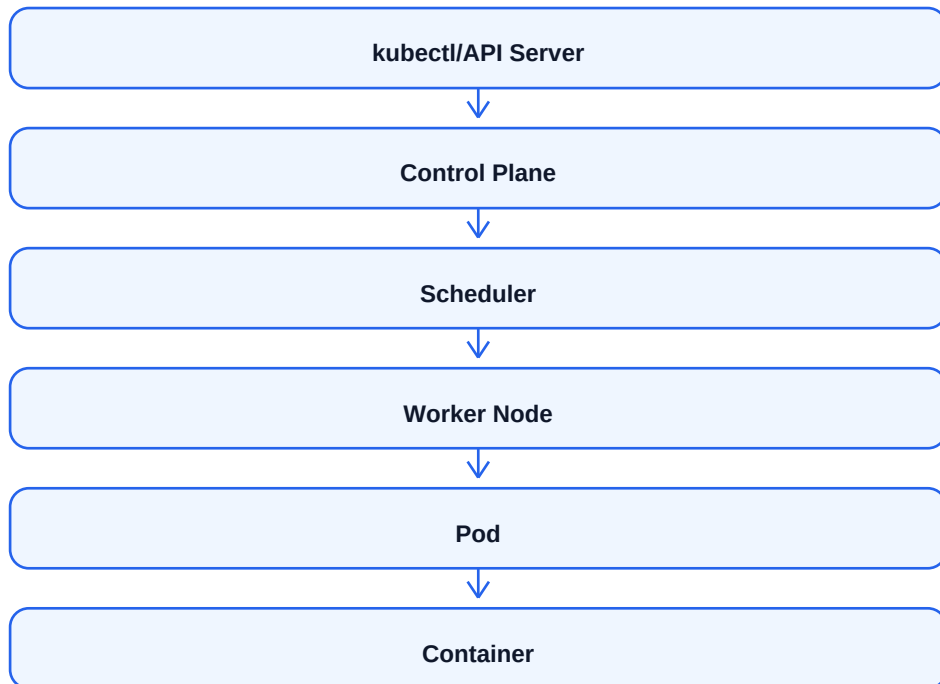
```
docker build -t myapp:1.0 .
docker run -p 8080:8080 myapp:1.0
docker ps
docker stop <container_id>
docker tag myapp:1.0 username/myapp:1.0
docker push username/myapp:1.0
```

4. Kubernetes Overview and Architecture

Kubernetes is an orchestration platform used to manage containers at scale. It automates deployment, scaling, load balancing, self-healing and rolling updates. Kubernetes is useful when many containers need to run across multiple machines.

Kubernetes architecture includes control plane and worker nodes. The control plane manages cluster state. Worker nodes run application containers inside Pods. A Pod is the smallest deployable unit in Kubernetes.

- API Server: entry point for cluster communication.
- Scheduler: assigns Pods to nodes.
- Controller Manager: maintains desired state.
- etcd: stores cluster configuration and state.
- Kubelet: agent running on worker nodes.
- Kube-proxy: handles networking rules.
- Pod: smallest deployable unit.
- Service: stable network access to Pods.



5. Kubernetes Deployment Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: username/myapp:1.0
          ports:
            - containerPort: 8080
```

6. HELM and Kubernetes Ecosystem

Helm is a package manager for Kubernetes. It uses charts to define, install and upgrade Kubernetes applications. A Helm chart contains templates and values files. Helm simplifies deployment of complex applications such as databases, monitoring stacks and microservices.

- Minikube or kind can be used to install Kubernetes locally.
- Kubernetes Dashboard provides web UI for cluster management.
- Helm charts package Kubernetes manifests.
- Ecosystem tools include Prometheus, Grafana, Istio, Argo CD and Ingress controllers.

Important Semester Exam Questions

Unit 1

- Define DevOps and explain its benefits.
- Differentiate CI, Continuous Delivery and Continuous Deployment.
- Explain Infrastructure as Code.
- Explain DevOps lifecycle with diagram.
- Compare DevOps and traditional operations.

Unit 2

- Explain DevOps process and framework.
- Explain source code management and code review.
- Explain build, artifact and release management.
- Explain DevOps maturity lifecycle and checklist.
- Explain Agile framework and cloud as foundation.

Unit 3

- Explain CI/CD best practices.
- Explain Jenkins setup and GitHub integration.
- Explain Maven integration with Jenkins.
- Write Jenkins pipeline example.
- Explain Gerrit and testing with Jenkins.

Unit 4

- Explain build pipeline and continuous deployment.
- Explain Chef components and Chef Automate workflow.
- Explain Ansible playbooks, modules, inventory and vault.
- Explain Nagios and Splunk monitoring.
- Write cloud security issues in DevOps.

Unit 5

- Explain Docker and containers.
- Write Dockerfile and Docker commands.
- Explain Kubernetes architecture.
- Explain Kubernetes dashboard and local installation.
- Explain Helm and Kubernetes ecosystem.